

```
.env .gitignore X main.dart app_config.dart
.gitignore
56 .flutter-plugins-dependencies
57 .metadata
58
59 # Android
60 android/.gradle/
61 android/app/debug
62 android/app/profile
63 android/app/release
64
65 # iOS
66 ios/Pods/
67 ios/.symlinks/
68 ios/Flutter/Flutter.framework
69 ios/Flutter/Flutter.podspec
70
71 # VS Code
72 .vscode/
73
74 # IntelliJ / Android Studio
75 .idea/
76 *.iml
77
78 # Mac
79 .DS_Store
80
81 # Logs
82 *.log
83
84 # Environment variables
85 .env You, 22 hours ago • commit practica2.5 ...
```

```
.env .gitignore main.dart app_config.dart X w
lib > config > app_config.dart > AppConfig > baseUrl
You, 22 hours ago | 1 author (You)
1 import 'package:flutter_dotenv/flutter_dotenv.dart';
2
You, 22 hours ago | 1 author (You)
3 class AppConfig {
4     static String get apiKey =>
5         dotenv.env['OPENWEATHER_API_KEY'] ?? '';
6
7     static String get baseUrl =>
8         dotenv.env['OPENWEATHER_BASE_URL'] ??
9         'https://api.openweathermap.org/data/2.5/weather';
10
11     static bool isConfigured() {
12         return apiKey.isNotEmpty && baseUrl.isNotEmpty;
13     }
14 }
```

```

factory Weather.fromJson(Map<String, dynamic> json) {}

// Validaciones principales
if (!json.containsKey('main') || !json.containsKey('weather')) {
    throw const FormatException('Respuesta API incompleta');
}

final weatherList = json['weather'];

if (weatherList is! List || weatherList.isEmpty) {
    throw const FormatException('Sin datos de clima');
}

final tempValue = json['main']['temp'];
    You, 22 hours ago • commit practica2.5
if (tempValue is! num) {
    throw const FormatException('Temperatura inválida');
}

return Weather(
    city: json['name'] ?? 'Desconocido',
    temp: tempValue.toDouble(),
    condition: weatherList[0]['main'] ?? 'Desconocido',
    description: weatherList[0]['description'] ?? '',
    unit: '°C',
    humidity: (json['main']['humidity'] ?? 0) as int,
    windSpeed: ((json['wind']['speed'] ?? 0).toDouble(),
);
}

Map<String, dynamic> toJson() => {
    'city': city,
    'temperature': temp,
    'condition': condition,
    'description': description,
    'humidity': humidity,
    'windSpeed': windSpeed,
};

```

```

import 'dart:async';
import 'dart:convert';
import 'dart:io';

import 'package:http/http.dart' as http;

import '../config/app_config.dart';
import '../models/weather.dart';

class WeatherService {
  static const Duration _timeout = Duration(seconds: 10);

  // Reutilizar el cliente HTTP es una buena práctica para mejorar el rendimiento
  final http.Client _httpClient;

  WeatherService({http.Client? httpClient}) : _httpClient = httpClient ?? http.Client();

  /// Obtiene el clima actual de una ciudad específica.
  Future<Weather> getWeather(String city) async {
    final cleanCity = city.trim();

    // 1. Validaciones previas
    if (cleanCity.isEmpty) {
      throw ArgumentError('La ciudad no puede estar vacía');
    }

    if (!AppConfig.isConfigured()) {
      throw Exception('API key no configurada. Revisa el archivo .env');
    }

    // 2. Construir la URL de forma segura usando Uri (evita problemas con espacios y caracteres especiales)
    final uri = Uri.parse(AppConfig.baseUrl).replace(
      queryParams: {
        'q': cleanCity,
        'appid': AppConfig.apiKey,
        'units': 'metric',
        'lang': 'es',
      },
    );
  }
}

```

```

    try {
        // 3. Ejecutar la petición HTTP
        final response = await _httpClient.get(uri).timeout(_timeout);

        // 4. Manejar códigos de respuesta
        return _handleResponse(response, cleanCity);

    } on SocketException {
        throw const HttpException('Sin conexión a internet');
    } on TimeoutException {
        throw TimeoutException('Tiempo de espera agotado. Intenta de nuevo');
    } on FormatException catch (e) {
        throw FormatException('Respuesta inesperada de la API: $e');
    }
}

/// Consulta el clima de varias ciudades en paralelo.
Future<List<Weather>> getWeatherForCities(List<String> cities) async {
    final futures = cities.map((city) => getWeather(city));
    return Future.wait(futures);
}

/// Manejo estructurado de las respuestas HTTP
Weather _handleResponse(http.Response response, String city) {
    switch (response.statusCode) {
        case 200:
            final json = jsonDecode(response.body) as Map<String, dynamic>;
            return Weather.fromJson(json);
        case 401:
            throw const HttpException('API key inválida o no activada aún');
        case 404:
            throw HttpException('Ciudad "$city" no encontrada');
        case 429:
            throw const HttpException('Límite de llamadas excedido. Espera un momento');
        default:
            throw HttpException('Error del servidor: ${response.statusCode}');
    }
}

void dispose() {
    _httpClient.close();
}
}

```

You, 22 hours ago | 1 author (You)

```
class Weather {
    final String city;
    final double temp;
    final String condition;
    final String description;
    final String unit;
    final int humidity;
    final double windSpeed;

    Weather({
        required this.city,
        required this.temp,
        required this.condition,
        required this.description,
        required this.unit,
        required this.humidity,
        required this.windSpeed,
    });

    factory Weather.fromJson(Map<String, dynamic> json) {
        // Validaciones principales
        if (!json.containsKey('main') || !json.containsKey('weather')) {
            throw const FormatException('Respuesta API incompleta');
        }

        final weatherList = json['weather'];

        if (weatherList is! List || weatherList.isEmpty) {
            throw const FormatException('Sin datos de clima');
        }

        final tempValue = json['main']['temp'];

        if (tempValue is! num) {
            throw const FormatException('Temperatura inválida');
        }

        return Weather(
            city: json['name'] ?? 'Desconocido',
            temp: tempValue.toDouble(),
            condition: weatherList[0]['main'] ?? 'Desconocido',
            description: weatherList[0]['description'] ?? '',
            unit: '°C',
            humidity: (json['main']['humidity'] ?? 0) as int,
            windSpeed: ((json['wind']['speed']) ?? 0).toDouble(),
        );
    }
}
```



5:52 ⓘ ⚙️ 🔒



Clima Actual



🔍 adasd

Buscar



HttpCiudad 'adasd' no encontrada

Reintentar

Integración BLE – Wearable

Sin conexión BLE

🔍 Buscar dispositivos BLE



Reloj_Prueba_BLE

UUID: 0000180a-0000-1000-8000-00805f9b34fb →

5:53 ⓘ ⚙️ 🔒



Clima Actual



🔍 berlin

Buscar

32.3°C

Berlin



Clear

Humedad: 30% | Viento: 4.5 m/s

🌡️ Cambiar Unidad (°C / °F)

Buscar Ciudades

Integración BLE – Wearable

Sin conexión BLE

🔗 Buscar dispositivos BLE

Beleto, Duque, BLE

